- Official Stripe.js Docs: https://stripe.com/docs

---

1. الـ Idempotency Keys (الضمان ضد التكرار)

دي أهم تقنية في الـ FinTech. تخيل السيرفر بتاعك بعت طلب لـ Stripe والنت قطع في النص؛ هل تبعت الطلب تاني؟ لو بعته ممكن تخصم من اليوزر مرتين.

- **الحل التقني:** Stripe بيستخدم Header اسمه `Idempotency-Key`.
- **اللوجيك:** لو بعت نفس الـ Key مرتين خلال 24 ساعة، Stripe مش بينفذ العملية التانية، وبيرجعلك نفس الـ Response بتاع العملية الأولى. ده بيضمن **Deterministic Behavior**.

---

2. الـ Webhook Event-Driven Architecture

ماتتعاملش مع الـ `success_url` كأنه مصدر الحقيقة (Source of Truth). ده مجرد Redirect لليوزر.

الـ Flow التقني للـ Webhook:

1. **Asynchronous Notification:** بمجرد ما البنك يوافق، Stripe بيبعت `POST Request` للسيرفر بتاعك.
2. **Signature Verification:** الـ Header بيبقى فيه `Stripe-Signature`. إنت لازم تستخدم الـ **Signing Secret** والـ Stripe Node Library عشان تعمل **Verification** للداتا. ده بيمنع أي حد (Hacker) إنه يبعت Request مزيف للسيرفر بتاعك ويدعي إنه Stripe.

JavaScript

```
// مثال للـ Verification
const event = stripe.webhooks.constructEvent(req.body, sig, endpointSecret);
```

---

3. الـ Stripe Objects Lifecycle

الـ Checkout Session مش Object معزول، ده "Parent" لـ Objects تانية:

- **PaymentIntent:** ده الـ Object نفسها "عملية الدفع" (من أول المسؤول عن تتبع دورة حياة `requires_payment_method` لحد `succeeded`).
- **Customer:** لو فعلت الـ `customer_creation` في الـ Session، نود أوتوماتيك بتكريت وتقدر Customer ID تربطه بالـ Database بتاعتك للعمليات اللي جاية (Recurring Payments).

4. الـ "Checkout Session "State Machine

الـ Session بتمشي بـ States محددة، وإنت "بترقبها" من الـ Webhooks:

1. `open` : اليوزر لسه في صفحة الدفع.
2. `complete` : الدفع نجح والداتا اتلمت.
3. `expired` : اليوزر قفل الصفحة ومكملش الدفع (الديفولت بعد 24 ساعة).

---

5. الـ PCI Compliance & Tokenization

ليه إنت مش بتلمس رقم الفيزا؟

- **Tokenization:** لما اليوزر بيكتب بياناته في صفحة Stripe، الداتا بتروح لـ Stripe مباشرة، وهما بيبدلوها بـ **Token** (مشفر String عبارة عن).
- **Scope:** بالطريقة دي، السيرفر بتاعك "Out of Scope" للـ PCI DSS (قواعد تأمين بيانات الكروت العالمية). إنت بس بتاع العملية، مش رقم الكارت ID بتتعامل مع الـ.

---

6. التعامل مع الـ Race Conditions

لو السيرفر بتاعك بطيء في معالجة الـ Webhook، وبعت لليوزر إيميل تأكيد، وبعدين الـ Webhook وصل تاني (Stripe بيعمل Retry لو السيرفر ماردش بـ 200 OK).

- **Technical Fix:** لازم تخلي الـ Webhook Handler بتاعك **Idempotent**. يعني قبل ما تفضي الكارت، تأكد في الداتابيز: `if (order.status === 'paid') return;`.

---

1. الـ Dynamic Tax Calculation

بدل ما تبعت رقم ثابت للضريبة، إنت بتفعل "المحرك" (Engine) بتاع Stripe.

- **الـ Registration:** لازم الأول في الـ Stripe Dashboard تحدد إنت مسجل ضريبياً في أنهي دول/ولايات.
- **الـ Automatic Engine:** بمجرد ما اليوزر يكتب عنوانه في صفحة الـ Checkout، الـ API بيبعت العنوان لـ Tax Engine، اللي بيرد بـ **Tax Amount** دقيق جداً بناءً على قوانين اللحظة دي في المكان ده.

---

2. الـ Tax Code (تصنيف المنتج)

مش كل المنتجات ضريبتها واحدة (الأكل غير الـ Software غير الكتب).

- **المفهوم:** كل `line_item` بتبتعه، لازم تحدد له `tax_code`.
- **مثال تقني:** لو بتبيع "E-book"، الكود بتاعه `txcd_10000000`.
- **ليه؟** عشان فيه دول بتعفي الكتب من الضريبة أو بتقللها، فـ Stripe محتاج يعرف "نوع" اللي بتبيعه عشان يحسب صح.

---

3. الـ Logic Integration في الـ Session

عشان تشغل الـ Taxes في الكود بتاعك، بتضيف الـ Parameters دي في الـ `stripe.checkout.sessions.create`:

JavaScript

```
automatic_tax: {
  enabled: true, // تفعيل المحرك التلقائي
},
```

---

4. الـ Inclusive vs Exclusive Taxes

دي حتة بتلخبط المحاسبين:

- **Inclusive (شاملة):** السعر اللي إنت عارضه (مثلاً 100 ضريبة $14 دي فيه) *ةبيرضلالمامش. StripeهيحسابلـTaxويقولكمانلـ100".*
- **Exclusive (إضافية):** السعر 100 *وStripeهيضيفعليهالضريبةفيالـCheckout(الزوينعفدي114).*

---

5. الـ Tax ID Collection (للمحترفين)

لو العميل بتاعك شركة (B2B)، بيبقى ليهم رقم تسجيل ضريبي عشان يخصموا الضريبة دي لاحقاً.

- **الـ Validation:** Stripe مش بس بياخد الرقم، ده بيكلم السيرفرات الحكومية (زي منظومة الضرائب المصرية أو الأوروبية) ويتأكد إن الرقم ده **Active** وسليم قبل ما يخصم الضريبة.

---

6. الـ Reporting (ما بعد الدفع)

لما العملية بتخلص، Stripe بيكريت **Tax Transaction Object**.

- **الـ Webhooks:** بيجيلك حدث اسمه `tax.settings.updated` لو قوانين الضريبة اتغيرت.
- **الـ Export:** في آخر السنة، تقدر تنزل شيت Excel فيه كل الضرايب اللي اتلمت متقسمة حسب الدولة، وتقدمها لمصلحة الضرائب وأنت حاطط رجل على رجل.

---

1. الـ Product Catalog (إدارة الأصول)

بدل ما تبعت بيانات المنتج (الاسم، السعر، الصورة) يدوي في كل Session، إنت بتسجلهم عند Stripe كـ **Objects**.

- **الـ Objects الهرمية:**
- **Product Object:** بيمثل السلعة نفسها (مثلاً: "كورس نود جي إس"). ده ثابت مش بيتغير كتير.
- **Price Object:** مثلاً: اشتراك شهري بـ 10) Price بيمثل "عرض السعر". المنتج الواحد ممكن يكون ليه كذا 100ـبـيونس وأ،).
- **ليه دي صياغة تقنية؟**

1. **Performance:** الـ Session بتبقى خفيفة جداً، إنت بتبعت بس الـ `price_id`.
2. **Dashboard Control:** Stripe تقدر تغير سعر المنتج من الداشبورد بتاعة فوراً، والكود بتاعك هيحدث نفسه أوتوماتيك من غير ما تلمس سطر واحد.
3. **Analytics:** Stripe بيقدر يطلع لك تقارير مبيعات دقيقة لكل منتج بالاسم والكود.

---

2. الـ Dynamic Updates (التحديث اللحظي)

دي بقى قمة التكنولوجيا في تجربة المستخدم. تخيل اليوزر وهو في صفحة الـ Checkout، اختار "شحن سريع" بدل "عادي"، إزاي السعر يتحدث قدامه من غير ما يعمل Refresh للصفحة؟

- **الـ Logic:** إنت بتعرف الـ Session إنها قابلة للتعديل بناءً على اختيارات اليوزر.
- **الـ Shipping Options:** بدل ما تبعت سعر شحن ثابت، إنت بتبعت لستة `shipping_options`.
- **الـ Server-side Validation:** لما اليوزر بيغير اختياره (مثلاً يغير عنوانه لدولة تانية)، Stripe بيبعت طلب لسيرفرك (Webhook) يسأله: "هل متاح شحن للمكان ده؟ وبكام؟". سيرفرك بيرد بالداتا الجديدة، والـ UI بتاع Stripe بيتحرك قدام اليوزر "سيملس" (Seamless).

---

3. الـ Discount Codes & Coupons (نظام الخصومات)

اللينك بيوضح إزاي تفعل حقل "Promo Code".

- `allow_promotion_codes: true` : السطر ده بيفتح خانة في الـ UI.
- **التحقق:** Stripe هو اللي بيتأكد إن الكود سليم، مش منتهي الصلاحية، ومطابق للشروط (مثلاً خصم 10% لو المشتريات فوق 50$). إنت كدة رميت حمل الـ Discount Logic كله عليهم.

4. الـ Upsells (زيادة المبيعات)

دي ميزة تقنية بتزود فلوسك. تقدر تضبط الـ Session بحيث لو اليوزر بيشتري "موبايل"، Stripe يعرض عليه "جراب" كـ **Upsell** في نفس صفحة الدفع.

- **الـ Logic:** لو اليوزر وافق، Stripe بيحدث الـ `line_items` أوتوماتيك ويضيف السعر الجديد.

---

5. الـ Cross-object References

تقنياً، إنت بتقدر تربط الـ `customer_id` بالـ `product_id` بالـ `price_id` .

- لما بتستخدم الـ Catalog، الـ Webhook اللي بيرجعلك بيبقى فيه كل الـ IDs دي. ده بيخلي الـ Data Mapping في قاعدة البيانات بتاعتك (MongoDB مثلاً) سهل جداً لأنك بتتعامل مع IDs ثابتة مش Strings متغيرة.

---

1. الـ Subscription Lifecycle (دورة حياة الاشتراك)

الاشتراك مش `PaymentIntent` واحد، ده "ماكينة" بتولد عمليات دفع أوتوماتيك.

- **الـ `status` :** الاشتراك بيبدأ بـ `incomplete` (مستني أول دفعة)، أول ما تنجح بيتحول لـ `active` . لو الفيزا انتهت أو الفلوس خلصت، بيتحول لـ `past_due` وبعدين `canceled` .
- **الـ Billing Cycle:** Stripe هو اللي بيحسب ميعاد الدفعة الجاية أوتوماتيك بناءً على الـ `interval` (شهر/سنة) اللي إنت حددته في الـ **Price Object**.

---

2. الـ "Price" هو المحرك الأساسي

في الـ Subscriptions، إنت **إجباري** تستخدم الـ **Product Catalog** اللي شرحناه.

- إنت بتبعت الـ `price_id` . الـ Price ده هو اللي شايل الـ Logic:
- **Interval:** (day, week, month, year).
- **Usage Type:** (Licensed: سعر ثابت، أو Metered: الدفع حسب الاستهلاك).

---

3. الـ Integration via Checkout (الطريقة الأسهل)

بدل ما تبني نظام إدارة اشتراكات معقد، بتستخدم `mode: 'subscription'` جوه الـ Checkout Session:

JavaScript

```
const session = await stripe.checkout.sessions.create({
  mode: 'subscription', // أهم سطر هنا
  payment_method_types: ['card'],
  line_items: [{
      price: 'price_1PQ...', // ID السعر المتكرر من الكتالوج
      quantity: 1,
  }],
  success_url: '...',
  cancel_url: '...',
});
```

---

4. الـ Webhooks: "روح" الاشتراكات

في البيع العادي (One-time)، الـ Webhook بيوصلك مرة واحدة. في الاشتراكات، الـ Webhooks هي اللي بتدير موقعك:

- `customer.subscription.created` : أول ما يشترك.
- `invoice.paid` : كل شهر لما Stripe يسحب الفلوس بنجاح، بيبعتلك الـ Event ده عشان تمد فترة الصلاحية لليوزر في الداتابيز.
- `invoice.payment_failed` : لو السحب فشل، هنا بتبعت إيميل لليوزر أو تقفل حسابه.

---

5. الـ Customer Portal (توفير مجهود جبار)

اليوزر يقدر من خلالها (Hosted Portal) بيديك صفحة جاهزة Stripe:

- يكنسل الاشتراك (Cancel).
- يغير نوع الاشتراك (Upgrade/Downgrade).
- يحدث بيانات الفيزا. **تقنياً:** إنت بتبعت اليوزر للينك الـ Portal ده، وStripe بيقوم بالباقي ويرجعهولك لما يخلص.

---

6. الـ Proration (حسابات الفروق)

لو يوزر مشترك في الخطة الـ "Basic" بـ 10$ وقرر في نص الشهر يحول للـ "Pro" بـ 30$.

- Stripe بيعمل **Proration Logic**: بيحسب هو استهلك قد إيه من الـ 10$ ويخصمها من الـ 30$ الجديدة ويحاسبه على الفرق بس. إنت مش محتاج تعمل عملية حسابية واحدة!

---

1. الـ Price Localization (الأسعار المرنة)

الفكرة هنا مش مجرد تحويل عملة (Currency Conversion)، دي استراتيجية كاملة اسمها **Multi-currency**.

- **الـ Stripe Adaptive Pricing:** عنده محرك ذكي بيغير العملة بناءً على "مكان العميل" (IP Address). لو الزبون فتح من مصر هيشوف السعر بالـ **EGP**، ولو فتح من أمريكا هيشوفه بالـ **USD**.
- **ثبات السعر (Fixed Prices per Currency):** بدل ما تسيب الـ Exchange Rate يغير السعر كل يوم، إنت بتقدر تحدد سعر ثابت لكل عملة في الـ **Price Object**.
  - *مثال تقني:* المنتج بـ $10 في أمريكا، بس إنت عايزه بـ 300 جنيه في مصر (حتى لو الدولار اتغير). Stripe هيعرض الـ 300 جنيه للزبون المصري بناءً على الـ `Price ID` المرتبط بالعملة دي.
- **الـ Zero-decimal Currencies:** خلي بالك وأنت بتكود؛ فيه عملات زي الـ **JPY** (الين الياباني) مفيهاش "قروش" (Cents). فلو السعر 100 ين، بتبعته 100 مش 10000. Stripe Tax و Checkout بيتعاملوا مع ده أوتوماتيك لو حددت العملة صح.

---

2. الـ Promotions (أكواد الخصم والـ Coupons)

فيه فرق تقني كبير بين الـ **Coupon** والـ **Promotion Code**:

- **الـ Coupon (الكوبون):** ده الـ Object الأساسي اللي فيه "قواعد الخصم" (مثلاً: خصم 20%، أو خصم 50 جنيه، صالح لمدة 3 شهور). ده اليوزر مش بيشوفه ككلمة، ده "حاوية" للقواعد.
- **الـ Promotion Code:** ده "الكلمة" اللي اليوزر بيكتبها (مثلاً: `EID_2026`). الكود ده بيبقى مربوط بـ Coupon معين.
- **الـ Technical Constraints (القيود):**
  - **`first_time_transaction`:** تقدر تخلي الكود يشتغل بس لأول مرة اليوزر يشتري فيها.
  - **`minimum_amount`:** الكود ميتفعلش غير لو السلة فيها أكتر من 500 جنيه.
  - **`expires_at`:** تحديد وقت انتهاء الكود بالثانية (Unix Timestamp).

---

3. الـ Implementation (إزاي تفعلهم في الكود؟)

عشان تشغل الـ Promotion Codes في الـ Session بتاعتك، الموضوع تافه تقنياً بس مفعوله جبار:

JavaScript

```
const session = await stripe.checkout.sessions.create({
  payment_method_types: ['card'],
  line_items: [...],
  mode: 'payment',
  allow_promotion_codes: true, // السطر ده بيظهر خانة الـ Promo Code في الـ UI
  // بيقوم بالمهمة بناءً على الـ Price IDs اللي بتبعتها، Stripe بالنسبة للـ Localization
```

```
  success_url: '...',
  cancel_url: '...',
});
```

---

4. الـ Webhooks (متابعة الخصم)

لما اليوزر بيستخدم كود خصم، الـ `checkout.session.completed` بيبعتلك بيانات الخصم جوه أوبجيكت الـ `total_details`:

- `amount_discount` : الرقم اللي اتخصم فعلياً.
- `discounts` : مصفوفة (Array) فيها الـ IDs بتاعة الـ Coupons اللي استخدمت.

> **تريكة تقنية:** لازم تسجل الداتا دي في الـ Database عندك عشان لو حبيت تعمل "Refund" بعد كدة، تبقى عارف اليوزر دفع كام بالظبط بعد الخصم.

---

5. الـ Local Payment Methods

الـ Localization مش بس سعر، ده كمان "طريقة دفع". في اللينك بيوضح إن Stripe Checkout هيعرض لليوزر طرق الدفع المشهورة في بلده (زي **Fawry** في مصر، أو **iDEAL** في هولندا) أوتوماتيك لو إنت مفعلها في الـ Dashboard، من غير ما تغير حرف في كود الـ Node.js

---

1. الـ Setup Intent (حفظ الكارت بدون دفع)

اللينك الأول بيشرح إزاي تسجل كارت العميل من غير ما تسحب منه مليم دلوقتي.

- **السيناريو:** اليوزر بيضيف وسيلة دفع لحسابه عشان يشتري بيها بعدين (زي "أمازون").
- **الـ `mode` :** بتستخدم `'mode: 'setup` .
- **النتيجة التقنية:** Stripe بيكريت `SetupIntent` . بعد ما اليوزر يخلص، الـ Webhook هيبعتلك `SetupIntent.succeeded` ومعاه `payment_method_id` .
- **الزتونة:** إنت بتخزن الـ `payment_method_id` والـ `customer_id` عندك في الداتابيز. المرة الجاية لما يجي يشتري، مش هتفتحه صفحة الـ Checkout؛ إنت هتكلم Stripe من السيرفر فوراً وتقوله: "اسحب من الكارت اللي آخره 4242 اللي متسجل عندنا".

---

2. الـ Save During Payment (اضرب عصفورين بحجر)

اللينك التاني بيشرح إزاي تخلي اليوزر يدفع "الآن" وفي نفس الوقت يوافق إنك تخزن الكارت للمرات الجاية.

- **الـ Parameter:** بتضيف `payment_intent_data: { setup_future_usage: 'off_session' }`.
- **المفهوم التقني:** * **On-Session:** العميل فاتح الموقع دلوقتي وبيدفع.
- **Off-Session:** العميل مش موجود، وإنت بتسحب منه (زي اشتراك شهري أو تكملة حساب).
- **النتيجة:** Stripe بيعمل `PaymentIntent` وبيربطه بالـ `Customer` ويفعل خاصية الـ Reuse أوتوماتيك.

---

3. الـ After the Payment (مرحلة ما بعد "تم بنجاح")

اللينك التالت ده هو "خدمة ما بعد البيع" برمجياً. إيه اللي بيحصل بعد ما الـ `status` تبقى `complete`؟

أ. الـ Fulfillment (تنفيذ الطلب)

إنت بتعتمد على الـ (`checkout.session.completed`) Webhook. هنا بتنفذ الـ Business Logic بتاعك:

- تحديث المخزن (Inventory).
- إرسال إيميل التأكيد.
- فتح الصلاحيات (لو كورس أو اشتراك).

ب. الـ Post-payment Invoice (الفواتير)

تقدر تخلي Stripe يبعت فاتورة PDF احترافية لليوزر أوتوماتيك أول ما الدفع يخلص عن طريق تفعيل الـ `invoice_creation`.

ج. الـ Post-payment Page Customization

تقدر تعدل الـ "Success Page" اللي بتظهر عند Stripe عشان تعرض رسالة مخصصة أو زرار يرجع اليوزر لحتة معينة في موقعك (مثلاً: "Go to My Dashboard").

---

4. الـ Customer Portal (التحكم الكامل)

بعد الدفع، اليوزر بيحتاج يدير اشتراكاته أو يشوف فواتيره القديمة.

- بدل ما تبني صفحة "My Orders" وتوجع دماغك، Stripe بيديك الـ **Customer Portal**.
- **Technical Implementation:** إنت بتكريت `portal.sessions` وبتبعت اليوزر للينك بتاعها. هناك بيقدر يشوف الـ Payment History ويغير الكارت المتسجل.

---

5. الـ Refunds (المرتجع)

لو اليوزر طلب يرجع الفلوس، إنت بتستخدم الـ `payment_intent_id` اللي اتخزن وقت الدفع وتعمل `stripe.refunds.create()`.

- **التنبيه التقني:** الـ Refund بياخد من 5 لـ 10 أيام عمل عشان يرجع في حساب العميل، ولازم تسمع للـ Webhook `charge.refunded` عشان تحدث الداتابيز عندك.

---

1. الـ Stripe Elements (الـ UI المفصل)

بدل ما تبعت اليوزر لصفحة Stripe، إنت بتجيب "حتت" من Stripe وتحطها جوه موقعك (Components).

- **التقنية:** بتستخدم الـ `Payment Element`. ده بيحقن (Inject) فورم دفع "آمنة" جوه الـ Div بتاعك.
- **القوة:** إنت كدة بتحافظ على الـ Design System بتاعك بالملي، وفي نفس الوقت مفيش أي بيانات فيزا بتلمس السيرفر بتاعك (PCI Compliance محفوظ).
- **التحديث التلقائي:** الـ Elements دي ذكية؛ لو Stripe نزل طريقة دفع جديدة في بلد العميل، الـ Component هيحدث نفسه ويظهرها لليوزر من غير ما تلمس الكود.

---

2. الـ Payment Intents API (التحكم اليدوي)

دي هي "العمود الفقري" لأي عملية دفع متقدمة.

- **الـ Lifecycle:** إنت اللي بتدير الحالات يدوي:

1. `create`: بتعمل طلب دفع في السيرفر وبتاخد `client_secret`.
2. `confirm`: العميل بيأكد الدفع في الفرونت إند.
3. `capture`: إنت اللي بتسحب الفلوس فعلياً في الوقت اللي تحبه (اختياري).

- **الزتونة:** دي بتسمح لك تعمل "Complex Flows" زي إنك تأكد الدفع بس ما تسحبش الفلوس غير لما تتأكد إن المنتج متوفر في المخزن.

---

3. الـ Authorization and Capture (نظام الـ "Hold")

لو شغال في تطبيق زي (Uber) أو (Airbnb)، إنت مش بتسحب الفلوس فوراً.

- **اللوجيك:** بتعمل `capture_method: 'manual'`.
- **العملية:** Stripe بيعمل "بلوك" على المبلغ في كارت العميل (Authorization). معاك لحد 7 أيام تعمل `capture` (تسحب الفلوس) أو `cancel` (تفك الحجز).
- **الفائدة:** بتوفر على نفسك مصاريف الـ Refund لو العملية اتكنسلت، لأن الفلوس أصلاً مخرجتش من حساب العميل.

---

4. الـ 3D Secure & SCA (الأمان المتقدم)

في أوروبا ودول كتير، البنك لازم يطلب "OTP" أو "بصمة" عشان الدفع يكمل (Strong Customer Authentication).

- **التعامل البرمجي:** الـ SDK بتاعة Stripe (في الـ Web أو الـ Mobile) بتعرف أوتوماتيك لو العملية محتاجة تفعيل، وبتفتح الـ Popup بتاع البنك لليوزر.
- **الـ `next_action`:** لو السيرفر استلم حالة `requires_action` من Stripe، لازم تبعت الـ `client_secret` للفرونت عشان الـ SDK يتصرف.

---

5. الـ Stripe Radar (تخصيص الحماية)

في الليفل ده، إنت مش بتعتمد على حماية Stripe العامة بس.

- **الـ Rules:** تقدر تكتب "Logic" خاص بيك في الـ Dashboard: "ارفض أي عملية دفع من كارت صادر من دولة (X) لو المبلغ أكبر من 500$".
- **الـ Metadata:** ابعت `userId` و `ip_address` و `device_id` في الـ Metadata؛ الـ Radar بيستخدم الداتا دي عشان يبني بروفايل للمحتالين ويمنعهم قبل ما يضربوا الفيزا.

---

6. الـ Multi-party Payments (Stripe Connect)

لو بتعمل Marketplace (زي Amazon أو TaskRabbit) وعايز تقسم الفلوس.

- **الـ Flow:** الزبون بيدفع 100$ -> Stripe بياخد عمولته -> إنت (المنصة) بتاخد عمولتك -> والباقي بيروح لمحفظة "البائع" (Connected Account).
- **التعقيد:** إنت هنا بتدير "محافظ" (Wallets) وبتقرر مين اللي يشيل مصاريف الدفع (الـ Fees).

---

1. الـ الـ "Payment Sheet" (المنقذ)

بدل ما تقعد تبني فورم للفيزا وتوجع قلبك بالـ UI وتأمين البيانات، Stripe بيديك الـ **Payment Sheet**.

- **التعريف:** ده "Native UI Component" جاهز بيظهر من تحت الشاشة (BottomSheet).
- **القوة:** هو بيتعرف لوحده لو الموبايل عليه Apple Pay أو Google Pay وبيظهرهم كأول خيارات.
- **الـ Localization:** بيترجم لغة الفورمة والعملة أوتوماتيك بناءً على إعدادات موبايل العميل.

---

2. الـ Backend-to-Frontend Flow (الماسورة)

عشان الدفع يتم على الموبايل، لازم السيرفر بتاعك (Node.js) والموبايل (React Native/Flutter/Native) يرقصوا مع بعض:

1. **السيرفر:** بيكريت `PaymentIntent` ويرجع `client_secret` .
2. **السيرفر:** لو عايز اليوزر يحفظ الكارت، بيكريت `Customer` و `Ephemeral Key` (مفتاح مؤقت عشان الموبايل يقدر يوصل لبيانات العميل بأمان).
3. **الموبايل:** بياخد الـ 3 حاجات دول (`customerId` , `ephemeralKey` , `client_secret`) ويبدأ بيهم الـ `PaymentSheet` .

---

3. الـ Ephemeral Keys (سر الأمان)

دي حتة تقنية صايعة جداً في الموبايل.

- **المشكلة:** الموبايل "بيئة غير آمنة" مقارنة بالسيرفر. لو بعت الـ Secret Key بتاعك للموبايل، موقعك اتدمر.
- **الحل:** الـ **Ephemeral Key** هو مفتاح صلاحيته دقائق، بيدي الموبايل إذن "مؤقت" إنه يشوف كروت العميل ده بس. إنت بتطلبه من Stripe في السيرفر وتصبه في الموبايل.

---

4. الـ Apple Pay & Google Pay (الـ One-Tap)

أهم ميزة في الـ Mobile SDK هي الـ **Native Integration**.

- **Apple Pay:** لازم في الـ Xcode تفعل الـ "Apple Pay Capability" وتجيب `merchantIdentifier` من حساب Apple Developer وتربطه بـ Stripe.
- **Google Pay:** لازم تفعلها في الـ Google Play Console وتحدد الـ `countryCode` والـ `currencyCode` .
- **النتيجة:** اليوزر بيدفع ببصمة وشه في أقل من 3 ثواني.

---

5. الـ Custom Flow (لو عايز تفنن)

لو الـ Payment Sheet مش عاجباك وعايز تعمل Design خاص بيك 100%:

- بتستخدم الـ **Native Elements** (زي `CardField` في React Native).
- إنت اللي بتعمل الزرار، ولما اليوزر يدوس، بتنادي على ميثود `confirmPayment` وتعدي لها الـ `client_secret` .
- **التحذير:** دي بتاخد وقت أطول في الكود وبتحتاج مجهود أكبر في الـ UX.

---

6. الـ Webhooks (مرة تانية وعاشرة)

على الموبايل، الـ Webhooks أهم بمليون مرة من الويب.

- **السبب:** اليوزر ممكن يدفع، والعملية تنجح، وفي اللحظة دي جاله تليفون أو النت قطع أو البطارية فضيت. الـ App هيقفل والـ Callback مش هيرجع.
- **الحل:** السيرفر بتاعك لازم "يسمع" للـ Webhook عشان يحدث حالة الطلب في الداتابيز، حتى لو الموبايل بتاع العميل ولع!

(`stripeController.js`)

JavaScript

```
const stripe = require('stripe')(process.env.STRIPE_SECRET_KEY);

exports.createMegaCheckoutSession = async (req, res, next) => {
    try {
        const session = await stripe.checkout.sessions.create({
            // 1. نوع الدفع: اشتراك دوري (Subscriptions)
            mode: 'subscription',

            // 2. طرق الدفع: كروت، فوري، Apple Pay (بتتحدد من الـ Dashboard أو
الكود)
            payment_method_types: ['card'],

            // 3. المنتجات: بنبعت الـ Price IDs من الكتالوج بتاعك
            line_items: [
                {
                    price: 'price_1PQ_BASIC_PLAN_ID', // اشتراك أساسي
                    quantity: 1,
                },
                {
                    price: 'price_1PQ_ADDON_ID', // منتج إضافي (One-time
addon)
                    quantity: 2,
                }
            ],

            // 4. الضرائب التلقائية (Automatic Taxes)
            automatic_tax: { enabled: true },

            // 5. أكواد الخصم (Promo Codes)
            allow_promotion_codes: true,

            // 6. تجميع بيانات الشحن (Shipping)
```

```
        shipping_address_collection: {
            allowed_countries: ['EG', 'US', 'SA', 'AE'],
        },
        shipping_options: [
            { shipping_rate: 'shr_123_STANDARD_ID' }, // شحن عادي
            { shipping_rate: 'shr_456_EXPRESS_ID' }   // شحن سريع
        ],

        // 7. تجميع بيانات إضافية (Custom Fields & Phone)
        phone_number_collection: { enabled: true },
        custom_fields: [
            {
                key: 'tshirt_size',
                label: { type: 'custom', custom: 'مقاس التيشرت' },
                type: 'dropdown',
                dropdown: {
                    options: [
                        { label: 'Small', value: 's' },
                        { label: 'Large', value: 'l' }
                    ]
                }
            },
            {
                key: 'order_note',
                label: { type: 'custom', custom: 'ملحوظة خاصة' },
                type: 'text',
            }
        ],

        // 8. حفظ الكارت للمستقبل (Save for Future Use)
        payment_intent_data: { setup_future_usage: 'off_session' },

        // 9. الموافقات القانونية (Consent)
        consent_collection: {
            promotions: 'auto', // موافقة على إيميلات تسويقية
            terms_of_service: 'required', // لازم يوافق على الشروط
        },

        // 10. الـ Metadata (الخيط اللي بيربط Stripe بالـ DB بتاعتك)
        metadata: {
            userId: req.user._id.toString(),
            cartId: 'cart_999',
            couponSource: 'Summer_Campaign_2026'
        },

        // 11. الفواتير التلقائية (Invoice Creation)
```

```
            invoice_creation: { enabled: true },

            // 12. روابط العودة
            success_url: `${req.protocol}://${req.get('host')}/success?
session_id={CHECKOUT_SESSION_ID}`,
            cancel_url: `${req.protocol}://${req.get('host')}/cart`,
        });

        // بنبعت الـ URL لصفحة Stripe لليوزر عشان يتحول
        res.redirect(303, session.url);
    } catch (err) {
        next(err);
    }
};
```

---

2. الـ Webhook (المحطة اللي بتستلم "كل حاجة")

لما العميل يدفع، Stripe بيبعتلك الـ Object ده بكل الداتا اللي لميناها فوق. ده الكود اللي بيستلم "الزتونة":

JavaScript

```
exports.postStripeWebhook = async (req, res) => {
    const sig = req.headers['stripe-signature'];
    let event;

    try {
        event = stripe.webhooks.constructEvent(req.body, sig,
process.env.STRIPE_WEBHOOK_SECRET);
    } catch (err) {
        return res.status(400).send(`Webhook Error: ${err.message}`);
    }

    // التعامل مع الأحداث
    if (event.type === 'checkout.session.completed') {
        const session = event.data.object;

        // هنا بتسحب كل الداتا اللي اليوزر دخلها:
        const userEmail = session.customer_details.email;
        const shippingAddress = session.shipping_details.address;
        const customNote = session.custom_fields.find(f => f.key ===
'order_note').text.value;
        const userId = session.metadata.userId;
```

```
        // 1. سجل الـ Order في الـ DB
        // 2. ابعت إيميل التأكيد
        // 3. فعل الاشتراك لليوزر
        console.log(`Order for User ${userId} is READY!`);
    }

    res.json({ received: true });
};
```

---

3. ليه المثال ده هو "الخلاصة"؟ (The Technical Breakdown)

1. **الـ Full Compliance:** إنت هنا بتلم عنوان الشحن ورقم التليفون والموافقات القانونية من غير ما تعمل Form واحدة في موقعك.
2. **الـ Business Intelligence:** استخدام الـ `metadata` بيخليك عارف كل جنيه جاي منين ومين صاحبه في الداتابيز بتاعتك.
3. **الـ Subscription Logic:** بكلمة `'mode: 'subscription` إنت حولت السيستم بتاعك لـ SaaS بيسحب فلوس كل شهر أوتوماتيك.
4. **الـ Error Handling:** باستخدام الـ `success_url` مع الـ `{CHECKOUT_SESSION_ID}` ، تقدر في صفحة النجاح تنده على الـ API وتتأكد إن العملية تمت فعلاً قبل ما تظهر "شكراً لك".